

Aula 04/15

Escola Tecnológica FPFtech

Curso: Automação para Indústria 4.0

Módulo: IoT Aplicado à Manufatura

Prof. Denis Moraes Guimarães

02 de abril de 2026

Sumário

- ▶ Segurança utilizando criptografia
- ▶ Privacidade de dados
- ▶ Bancos de dados a aplicação
- ▶ Nuvem e aplicação

Segurança utilizando criptografia

```
1 #include <iostream>
2 #include <cstring>
3 #include <openssl/evp.h>
4 #include <openssl/rsa.h>
5 #include <openssl/pem.h>
6 #include <openssl/sha.h>
7
8 void exemplo_hash(std::string senha) {
9     std::string texto = senha;
10     unsigned char hash[SHA256_DIGEST_LENGTH];
11
12     SHA256((unsigned char*)texto.c_str(), texto.size(), hash);
13
14     std::cout << "SHA-256: ";
15
16     for(int i = 0; i < SHA256_DIGEST_LENGTH; i++)
17         printf("%02x", hash[i]);
18
19     std::cout << "\n\n";
20 }
```

Segurança utilizando criptografia

```

1  void exemplo_aes() {
2      EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
3
4      unsigned char key[33] = "01234567890123456789012345678901";
5      unsigned char iv[17] = "0123456789012345";
6
7      unsigned char plaintext[] = "Mensagem secreta";
8      unsigned char ciphertext[128];
9      unsigned char decrypted[128];
10
11     int len, ciphertext_len, decrypted_len;
12
13     // Criptografar
14     EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv);
15     EVP_EncryptUpdate(ctx, ciphertext, &len, plaintext, strlen((char*)
16         plaintext));
17     ciphertext_len = len;
18     EVP_EncryptFinal_ex(ctx, ciphertext + len, &len);
19     ciphertext_len += len;
20
21     std::cout << "AES Criptografado: ";
22     for(int i = 0; i < ciphertext_len; i++)
23         printf("%02x", ciphertext[i]);

```

Segurança utilizando criptografia

```
1  // Descriptografar
2  EVP_DecryptInit_ex(ctx, EVP_aes_256_cbc(),
3                      NULL, key, iv);
4  EVP_DecryptUpdate(ctx, decrypted, &len,
5                    ciphertext, ciphertext_len);
6  decrypted_len = len;
7  EVP_DecryptFinal_ex(ctx, decrypted + len, &
8                      len);
9  decrypted_len += len;
10
11  decrypted[decrypted_len] = '\0';
12  std::cout << "AES Descriptado: " << decrypted
13             << "\n\n";
14
15  EVP_CIPHER_CTX_free(ctx);
16 }
```

Segurança utilizando criptografia

```
1 void exemplo_rsa() {
2     RSA *rsa = RSA_generate_key(2048, RSA_F4, NULL, NULL);
3
4     unsigned char mensagem[] = "Mensagem RSA";
5     unsigned char criptografado[256];
6     unsigned char descriptografado[256];
7
8     int len = RSA_public_encrypt(strlen((char*)mensagem), mensagem,
9                                 criptografado, rsa, RSA_PKCS1_PADDING);
10
11     std::cout << "RSA Criptografado: ";
12     for(int i = 0; i < len; i++)
13         printf("%02x", criptografado[i]);
14     std::cout << "\n";
15
16     int decrypted_len = RSA_private_decrypt(len, criptografado, descriptografado,
17                                             rsa, RSA_PKCS1_PADDING);
18
19     descriptografado[decrypted_len] = '\0';
20     std::cout << "RSA Decriptado: " << descriptografado << "\n\n";
21     RSA_free(rsa);
22 }
```

Segurança utilizando criptografia

```
1 int main() {  
2     exemplo_hash("w2e3r4t5");  
3     exemplo_aes();  
4     exemplo_rsa();  
5     return 0;  
6 }
```

Privacidade de dados

```
1 #include <openssl/sha.h>
2 #include <iostream>
3 #include <iomanip>
4
5 std::string hash_sha256(const std::string& input) {
6     unsigned char hash[SHA256_DIGEST_LENGTH];
7
8     SHA256((unsigned char*)input.c_str(), input.size(), hash);
9
10    std::stringstream ss;
11    for(int i = 0; i < SHA256_DIGEST_LENGTH; i++)
12        ss << std::hex << std::setw(2) << std::setfill('0') << (int)hash[i];
13
14    return ss.str();
15 }
```


Privacidade de dados

```
1 std::string mask_cpf(const std::string& cpf) {
2     return "***.***." + cpf.substr(6, 3) + "-**"
3     ;
4 }
5 int randomize(int valor) {
6     return valor + (rand() % 200 - 100);
7 }
8 int main() {
9     std::string cpf = "12345678900";
10    std::cout << "CPF anonimizado HASH: " <<
11        hash_sha256(cpf) << std::endl;
12    std::cout << "CPF anonimizado Mask " <<
13        mask_cpf(cpf) << std::endl;
14    std::cout << "Numero randomico: " <<
15        randomize(500) << std::endl;
16 }
```

Bancos de dados a aplicação

```
1 #include <iostream>
2 #include <sqlite3.h>
3
4 int main() {
5     sqlite3 *db;
6     char *errMsg = nullptr;
7
8     if (sqlite3_open("sensors.db", &db)) {
9         std::cerr << "Erro ao abrir banco: " << sqlite3_errmsg(db) << "\n";
10         return 1;
11     }
12
13     std::cout << "Banco pronto!\n";
14
15     const char *create_table_sql =
16         "CREATE TABLE IF NOT EXISTS sensors ("
17         "id INTEGER PRIMARY KEY AUTOINCREMENT,"
18         "name TEXT NOT NULL,"
19         "value REAL,"
20         "unit TEXT,"
21         "timestamp DATETIME DEFAULT CURRENT_TIMESTAMP"
22         ");";
23
24     if (sqlite3_exec(db, create_table_sql, 0, 0, &errMsg) != SQLITE_OK) {
25         std::cerr << "Erro ao criar tabela: " << errMsg << "\n";
26         sqlite3_free(errMsg);
27     }
```

Bancos de dados a aplicação

```

1  const char *insert_sql =
2      "INSERT INTO sensors (name, value, unit) VALUES "
3      "('Temperatura', 25.3, 'C'),"
4      "('Umididade', 60.2, '%'),"
5      "('Pressao', 101.5, 'kPa');"
6
7  if (sqlite3_exec(db, insert_sql, 0, 0, &errMsg) != SQLITE_OK) {
8      std::cerr << "Erro ao inserir: " << errMsg << "\n";
9      sqlite3_free(errMsg);
10 }
11 const char *select_sql = "SELECT * FROM sensors;";
12 auto callback = [](void*, int argc, char **argv, char **colName) -> int {
13     for (int i = 0; i < argc; i++) {
14         std::cout << colName[i] << ": "
15             << (argv[i] ? argv[i] : "NULL") << " | ";
16     }
17     std::cout << "\n";
18     return 0;
19 };
20 if (sqlite3_exec(db, select_sql, callback, 0, &errMsg) != SQLITE_OK) {
21     std::cerr << "Erro ao consultar: " << errMsg << "\n";
22     sqlite3_free(errMsg);
23 }
24 sqlite3_close(db);
25 return 0;
26 }

```

Nuvem e aplicação

```
1 #include <curl/curl.h>
2 #include <iostream>
3 #include <string>
4
5 size_t write_callback(void* contents, size_t size, size_t nmemb, void* userp) {
6     ((std::string*)userp)->append((char*)contents, size * nmemb);
7     return size * nmemb;
8 }
9
10 int main() {
11     std::cout << "Enviando para Firebase..." << std::endl;
12
13     CURL* curl = curl_easy_init();
14     if (!curl) {
15         std::cerr << "Erro ao iniciar CURL" << std::endl;
16         return 1;
17     }
18
19     std::string json = R("{ \"termometro\": \"29.9\" })";
20     std::string response;
```

Nuvem e aplicação

```

1  curl_easy_setopt(curl, CURLOPT_URL, "https://aula-33497-default-rtdb.
    firebaseio.com/teste.json");
2  curl_easy_setopt(curl, CURLOPT_POSTFIELDS, json.c_str());
3
4  // Capturar resposta
5  curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_callback);
6  curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
7
8  // Certificado (IMPORTANTE)
9  curl_easy_setopt(curl, CURLOPT_CAINFO, "curl-ca-bundle.crt");
10
11 CURLcode res = curl_easy_perform(curl);
12
13 if (res != CURLE_OK) {
14     std::cerr << "Erro CURL: " << curl_easy_strerror(res) << std::endl;
15 } else {
16     std::cout << "Sucesso! Resposta do Firebase:\n" << response << std::endl
17         ;
18 }
19 curl_easy_cleanup(curl);
20 return 0;
21 }

```